exa | Developer workshop

Exa in *Personal Agents*

An open-source agent with memory, search, and a browser.

Jakub Hojsan · FDE Lead at Exa

Qwen · Exa · Kernel · Mem0 · Linq



Example conversation

Clone now.
Let the install run.

PREREQUISITES

Node.js 26.12.1+ with npm · Install nvm ↗ · A coding agent running locally

PUBLIC REPOSITORY github.com/JakubHojsan/aie-code-personal-agent ↗

```
git clone https://github.com/JakubHojsan/aie-code-personal-agent.git
cd aie-code-personal-agent
nvm install
nvm use
npm ci
cp .env.example .env
```

Open the repository in your coding assistant.

nvm reads .nvmrc. Already have Node 26.12.1+? Skip the nvm commands.

Copy the .env template ↗

What is Exa?

Search for AI agents

An independent web index
and a network of data partners.

Exa Connect Data partners

- Similarweb
- Crunchbase
- ZoomInfo
- Harmonic
- Jinko
- Particle
- Baselayer
- Fiber.ai
- Affiliate.com
- Financial Datasets
- Definitive Healthcare
- Faraday AI
- Intellizence
- Kernel
- OpenAlex
- Traject Data
- DataBento
- Alpha Vantage

Coding agents

Documentation, APIs,
and code examples

CURSOR Cognition

GTM enrichment

Company research
and account context

HubSpot

Personal agents

Places, research,
and everyday decisions

Today's workshop

Find the sources.

`POST /search`

Search indexed web and data sources.
Return evidence your model can use.

YOU PROVIDE

A natural-language query



YOU RECEIVE

Source URLs + relevant passages

EXAMPLE “Sushi omakase in San Francisco, around $150 per person.”

Used today: Search with highlights.

Search API documentation ↗

Read URLs you already have.

`POST /contents`

Extract readable content from a known set of URLs.

YOU PROVIDE

One or more URLs

→

YOU RECEIVE

Page text, highlights, or summaries

EXAMPLE “Read the restaurant pages I have already collected.”

Today: content comes from Search.

Contents API documentation ↗

Delegate the research.

`POST /agent/runs`

Give Exa a research task and the shape of the result you need.

YOU PROVIDE

A task + an output schema

→

YOU RECEIVE

Grounded research + structured results

EXAMPLE “Find companies matching these criteria, then enrich each one.”

Asynchronous research runs.

Agent API documentation ↗

Follow what changes.

`POST /monitors`

Repeat a search and receive newly discovered results by webhook.

YOU PROVIDE

A topic + a schedule

→

YOU RECEIVE

Updates delivered to your application

EXAMPLE “Tell my application when new local events are announced.”

Optional next step: proactive updates.

Monitors API documentation ↗

Fewer tokens.
More useful context.

RETRIEVE FOR THE TASK

Robin San Francisco omakase
official price range

POST /SEARCH

```
{
  "query": "...",
  "contents": {
    "highlights": true
  }
}
```

Less page text for the model to process.
More room for conversation and tool results.
Source URLs stay attached.

Highlights documentation ↗

1 selected result from a 5-result response — title · url · highlights

Robin · San Francisco ↗

robinomakase.com/location/robin-san-francisco/

> “Prices range from $119 to $219 depending on each guests unique preferences.”

The $150 budget is within the published range.
Confirm the experience, tax, tip, and availability.

Real Search capture · title edited, excerpt shortened.
The interpretation above is authored for the workshop.

The model can cite the evidence
and fetch more when the task needs it.

Partner data reaches agents in different ways.

ACCESS PATH 01

Through Search

Some partner-backed sources resolve into search results.

Query → `/search` → Sources + excerpts

Use the returned evidence in your own agent loop.

Search data index ↗

ACCESS PATH 02

Through Exa Agent

Attach partner capabilities to an Agent run through `dataSources`.

`/agent/runs` → Partner tools + web research

 Similarweb  Financial Datasets  Particle

Exa Connect ↗

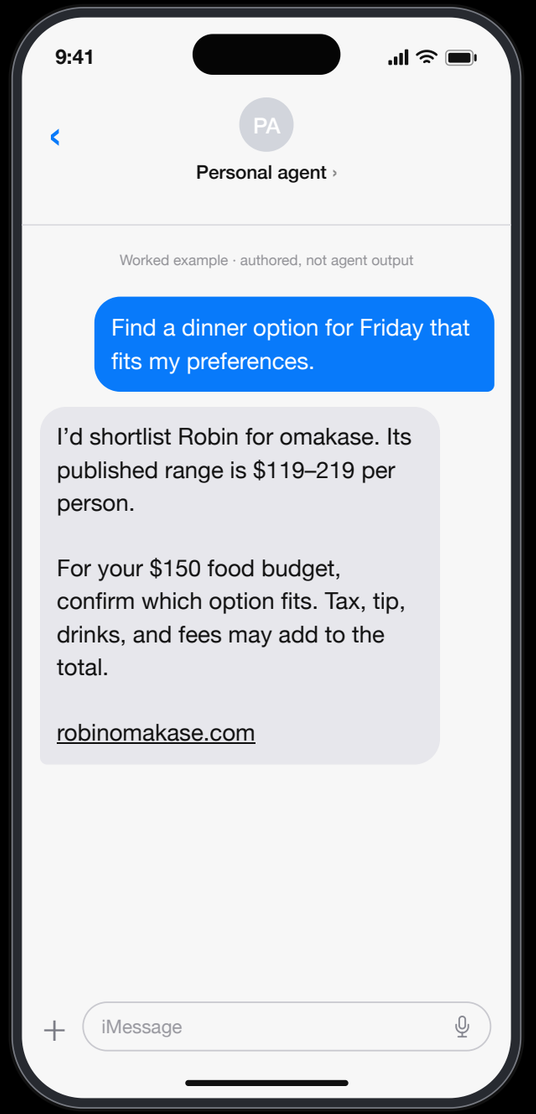
A table for two.

The same request, with your context.

EXAMPLE PREFERENCES

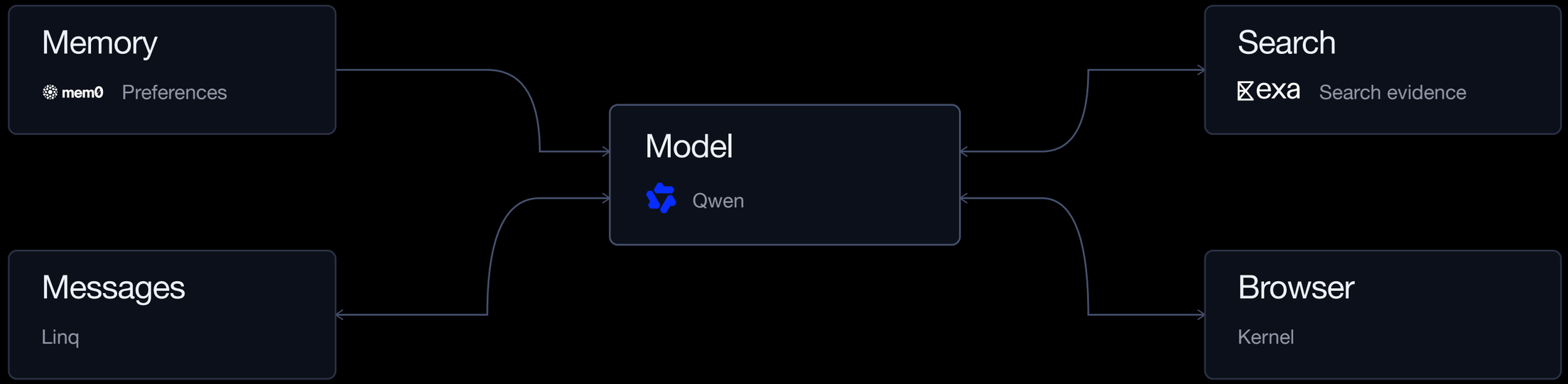
Sushi

$150 per person, before tax and tip



Authored example · official menu checked November 6, 2026

From message to answer.



Your laptop runs the agent.

ON YOUR LAPTOP

Agent loop

Conversation history

Mem0 storage

API calls

HOSTED CAPABILITIES

Qwen inference + embeddings

Exa search · Kernel browser

Linq messaging

The laptop and HTTPS tunnel stay running while you message the agent.

Now build your agent.

Start with Exa MCP, then connect the five integrations.

Model → Exa → Kernel → Mem0 → Linq

Start the workshop ↗

Your repository is ready. Connect Exa MCP,
then implement and verify one integration at a time.

Docs: Model ↗ Exa ↗ Kernel ↗ Mem0 ↗ Linq ↗

Connect Exa. Try one search.

01 / CONNECT

Open the setup page.

exa.ai/mcp ↗

Choose your coding assistant.
Follow its installation steps.
Reconnect and confirm Exa is enabled.

Remote server URL

```
https://mcp.exa.ai/mcp
```

02 / PASTE THIS PROMPT

```
Use Exa to find the official Search API docs.
Show how to request highlights.
Cite the official documentation URL.
Do not write code yet.
```

03 / CHECK You see an Exa tool call and a link to the official docs.

Get one real reply.

01 / PASTE INTO YOUR CODING ASSISTANT

```
Read starter/README.md and src/model.ts.
Implement TODO 1 in starter/integrations.ts.
Connect complete(); forward messages, tools, and signal.
Leave TODOs 2–5 unfinished. Run npm run typecheck.
Show how to verify a real reply with starter:chat.
```

Edit `starter/integrations.ts` · TODO 1

02 / CONFIGURE & RUN

`WORKSHOP_MILESTONE=1`
`OPENROUTER_API_KEY in .env`

```
npm run doctor
npm run starter:chat
```

03 / CHECK

A real model reply appears in your terminal.

TRY IT Help me plan a Friday sushi dinner in San Francisco.

Model docs ↗

Give it current sources.

01 / PASTE INTO YOUR CODING ASSISTANT

```
Use Exa MCP to find the official Search API docs.
Implement TODO 2 in starter/integrations.ts.
POST /search: type auto, numResults 5, highlights true.
Preserve URLs and retrieval time; bound the evidence.
Use the existing tool loop. Run npm run typecheck.
```

Edit `starter/integrations.ts` · TODO 2

02 / CONFIGURE & RUN

```
WORKSHOP_MILESTONE=2
EXA_API_KEY in .env

npm run doctor
npm run starter:chat
```

03 / CHECK

Open one returned URL and verify a claim.

TRY IT Find sushi in San Francisco under $150 per person before tax and tip. Include sources.

Exa API docs ↗

Read a page in a browser.

01 / PASTE INTO YOUR CODING ASSISTANT

```
Use Exa MCP to find the official Kernel browser docs.
Read src/browser.ts; implement TODO 3 only.
Connect browse() in starter/integrations.ts.
Forward URL, question, and signal unchanged.
Keep URL validation and cleanup. Run typecheck.
```

Edit `starter/integrations.ts` · TODO 3

02 / CONFIGURE & RUN

```
WORKSHOP_MILESTONE=3
KERNEL_API_KEY in .env

npm run doctor
npm run starter:chat
```

03 / CHECK

A real Kernel session returns the page's price text.

TRY IT Open https://www.robinomakase.com/location/robin-san-francisco/ and read the omakase price range.

Kernel docs ↗

Remember after a restart.

01 / PASTE INTO YOUR CODING ASSISTANT

```
Use Exa MCP to find the official Mem0 OSS docs.
Read src/memory.ts; implement TODO 4 only.
Return personalMemory(store) from connectMemory.
Keep owner scope, recall, and memory tools for explicit
requests.
Run typecheck; show a save → restart → recall check.
```

Edit `starter/integrations.ts` · TODO 4

02 / CONFIGURE & RUN

```
WORKSHOP_MILESTONE=4
Owner + embeddings configured
```

```
npm run starter:chat
```

03 / CHECK

Restart, ask what the agent remembers, then request dinner without repeating your preferences.

TRY IT Remember I live in San Francisco, love sushi, and budget $150 per person before tax and tip. What do you remember about me?

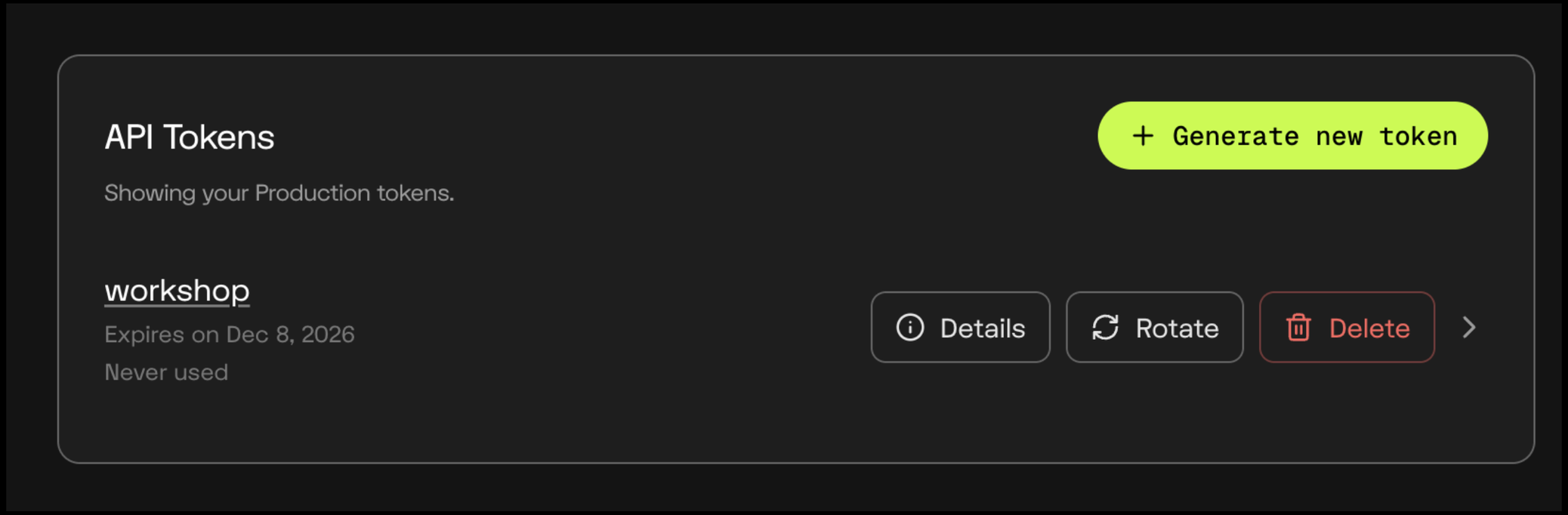
Mem0 docs ↗

Activate your sandbox. Generate your API token.

01 Open Linq sandbox ↗.
Verify your account for the 7-day trial.

02 Activate your sandbox.
Click Generate new token.

03 In your project folder, run `nano .env`.
Add `LINQ_API_KEY=your_token`.

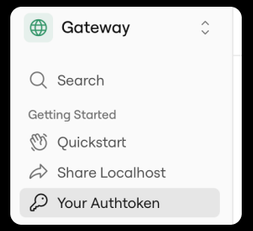


Choose the sandbox environment (screenshot: Production).

Linq setup guide ↗

Give Linq a URL to your laptop.

01 / START THE TUNNEL [Install ngrok ↗]



[Your Authtoken ↗]

Copy your token.
Authenticate once:

```
ngrok config add-authtoken YOUR_AUTHTOKEN
```

```
ngrok http 3400
```

Keep this terminal running.
Copy the HTTPS forwarding URL.

02 / ASK YOUR CODING AGENT

Your Linq API key can create the subscription.

```
Read docs/LINQ.md. Use LINQ_API_KEY from .env.
Get the HTTPS URL from ngrok at 127.0.0.1:4040/api/tunnels.
Create a Linq V3 webhook subscription via the API:
  target: HTTPS URL + /webhooks/linq?version=2026-02-03
  event: message.received
  phone filter: LINQ_PHONE_NUMBER from .env
Save signing_secret as LINQ_WEBHOOK_SECRET in .env.
Keep the subscription ID. Do not print any secrets.
Check for an existing subscription; avoid duplicates.
```

03 / CHECK The subscription exists and `LINQ_WEBHOOK_SECRET` is saved.

The agent starts after the secret is saved. Update Linq if the tunnel URL changes.

[Webhook guide ↗]

Text the agent on your laptop.

01 / PASTE INTO YOUR CODING ASSISTANT

```
Use Exa MCP to find the official Linq webhook docs.
Read docs/LINQ.md; implement TODO 5 only.
Connect sendReply using config.linqKey.
Forward chat, reply, and the idempotency key.
Keep the queue and signature checks. Run typecheck.
For now: sendReply(key, chat, reply, id, fetch, []).
```

Edit `starter/integrations.ts` · TODO 5

02 / RUN

```
WORKSHOP_MILESTONE=5
Keys + webhook saved in .env

npm run doctor
npm run starter:phone
```

03 / CHECK

Your phone receives a sourced reply.

iMessage + RCS compatible
SMS text fallback

TRY IT Text your provisioned line: “Find dinner for Friday using my saved preferences.”

Linq docs ↗

Setup guide ↗

From a URL to a weather card.

BEFORE / URL ONLY

iMessage

https://exa.ai/library/weather/san-francisco-california-united-states?latitude=37.7893&longitude=-122.4220&date=2026-11-07

→

AFTER / RICH PREVIEW IN IMESSAGE

iMessage



iMessage + RCS compatible · SMS: URL only

Exa Library weather ↗

Turn source links into cards.

01 / PASTE INTO YOUR CODING AGENT

```
Implement TODO 6 in starter/integrations.ts.
Extract public HTTPS URLs; prefer returned Exa Library URLs.
In deliverReply, pass sourceCards(text) to sendReply.
Send a short summary without raw URLs, then ONE OG card.
Use sourceCardLimit(request) for explicitly requested extras.
Keep plain texting: no bold, Markdown, or bullet lists.
Run typecheck and tests. Restart starter:phone.
Repeat the same search and compare before vs after.
```

Edit `starter/integrations.ts` · `TODO 6`

02 / RESTART THE PHONE AGENT

```
npm run starter:phone
```

Keep ngrok running.

03 / COMPARE

Repeat the same search.
Now see a source card
with a title and image
when the page provides them.

iMessage + RCS compatible

TRY IT Find sushi in San Francisco around $150 per person. Include sources.

Link-preview docs ↗

exa | Workshop 07 · Optional · 10–15 min

Reactions, then typing.

01 Choose a reaction

Jev picks an emoji that fits the message.
“No emojis” means no reaction.

RCS compatible*

02 Show the typing bubble

Start after the reaction.
Keep it visible while the agent works.

iMessage only

03 Send the reply

Clear typing when done—even if the request fails.

iMessage + RCS · SMS fallback

Your task

Complete TODO 7 and connect it to the starter server.

`starter/extras.ts`
`starter/server.ts`

Workshop guide ↗

In .env: `REACTION_MODE=jev`

Uses your OpenRouter key.

Try it

“Find sushi in San Francisco.”

Reaction → typing → reply

*Linq lists RCS reactions; custom emoji support needs a device check. Full setup and checks are in the coding prompt.

Let your agent speak.

01 / PASTE INTO YOUR CODING AGENT

```
Implement TODO 8 in starter/extras.ts.
Add voice routing in starter/server.ts.
Use src/extras.ts and docs/WORKSHOP.md as references.
Also recognize “send me a voice note” / “read that aloud”.
Generate a short Qwen answer, then call OpenRouter TTS:
  mistralai/voxtral-mini-tts-2603, en_paul_neutral, mp3.
Validate audio/mpeg; save the MP3 under .data/audio/.
Upload bytes to Linq; send attachment_id via /voicememo.
Persist audio and delivery state; prevent duplicate sends.
Run checks; restart starter:phone. Test /voice Say hello.
```

Edit `starter/extras.ts` · TODO 8

02 / RUN

```
npm run typecheck
npm test
npm run starter:phone
```

03 / CHECK

Native voice bubble: iMessage only.
Audio attachment: RCS compatible.
Then try “read that aloud”.

TRY IT Text: /voice Say hello in one sentence.

OpenRouter TTS docs ↗

Linq voice docs ↗

Your project's environment file.

In your project folder: `cp .env.example .env` → `nano .env`

Download .env.example ↗

ADD YOUR CREDENTIALS

```
OPENROUTER_API_KEY=
EXA_API_KEY=
KERNEL_API_KEY=
LINQ_API_KEY=
LINQ_WEBHOOK_SECRET=
LINQ_PHONE_NUMBER=
OWNER_PHONE_NUMBER=
```

Phone numbers include country codes.
Linq generates the webhook secret.

WORKSHOP DEFAULTS

```
WORKSHOP_MILESTONE=1
MODEL_NAME=qwen/qwen3.8-27b
MODEL_BASE_URL=https://openrouter.ai/api/v1
MEM0_EMBEDDING_MODEL=qwen/qwen3-embedding-4b
MEM0_EMBEDDING_DIMENSIONS=2560
AGENT_TIMEZONE=America/Los_Angeles
PORT=3400
DATA_DIR=.data
```

Start at milestone 1.
Increase to 5 as you finish each integration.